

University of Pittsburgh

Introduction to Operating Systems
CS/COE 1550

Spring 2022

Sherif Khattab

ksm73@pitt.edu

(Some slides are from **Silberschatz, Galvin and Gagne ©2013)**

Announcements

- Upcoming deadlines
  - Homework 1 is due today at 11:59 pm
  - Homework 2 is due next Monday at 11:59 pm
  - Lab 1 is due on 2/4 at 11:59 pm
  - Project 1 is due on 2/18 at 11:59 pm
- TA Office hours available on the syllabus page

Previous Lecture …

- Busy waiting problem and how to solve it using semaphores

Muddiest Points

- a little confused on the relationship between semaphores and spinlocks
- what's the advantage between a binary semaphore and a spinlock if a binary semaphore can only be for one process?
- What happens to each process with each up() and down() for multiple processes with varying priorities? Another walkthrough basically
- could you further explain how the semaphores with spinlocks still are able to keep their benefits?
  - What is the point of having semaphores if you are going to tie them with spinlocks? Does that not defeat the benefits of using a semaphore?
  - By introducing a spinlock within our semaphore implementation, doesn't that bring up the issue of busy waiting? Or is this not an issue because the critical section is a relatively small block of code?
- what is the different between swap and test&set?
- Do Semaphores have the same issue with reordering instructions that spinlocks have?

Muddiest Points (2/3)

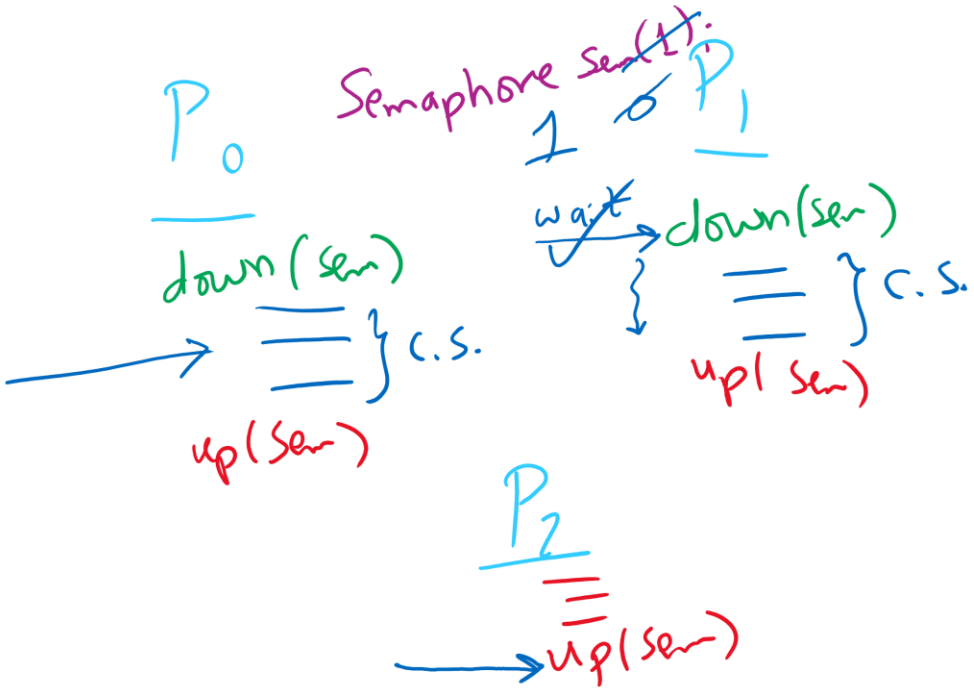
- where the lock() and unlock() go in the pseudo code of the spinlock in semaphores example
  - How to stop the up/down from breaking (the if else thing you brought up at the end)
- for the process list I am assuming we would use some sort of priority queue to sort the processes for the ProcessList?
- How a spinlock is implemented in a semaphore
- I think the functional difference between blocking and waiting. As in, what is actually happening while a process is infinite looping and awaiting a lock? Since it's not "blocked", does that mean that it prevents other processes from running, or can it be context switched out like any normal process?
- which implementation is most commonly used for semaphores
- difference between counting and binary semaphore (why counting would be used)
- can the semaphore be > 0, and if so what does that mean?
- How are spinlocks implemented in hardware? Are they the only locking mechanism implemented in hardware?

Muddiest Points (3/3)

- How process states are switched/clarification on ready/waiting/running
- What does the comment above the line of assembly code mean?
  - I'm curious about memory barriers. could you talk more about how to use _sync_synchronize() and how it works?
- Where exactly the code blocks
- from the tophat question: why can down move a process into ready
- When/why is reordering instructions in a critical section beneficial?
- Today, We heard "Block" a lot but it has the same meaning
- Implementation of mutex vs standard semaphore
- When would you know to use a spinlock instead of a semaphore?
- Why two context switches in semaphore
- What is/makes an operation atomic?

Semaphore Usage Problem: Compromising Mutual Exclusion

- Any process can up() the semaphore



- Solution: A **Mutex** can be up()'d only by the same process that down()'d it

Semaphore Usage Problem: Deadlock and Starvation

- Deadlock: two or more processes are waiting indefinitely for an event that can only be caused by a waiting process
  - P0 gets A, needs B
  - P1 gets B, needs A
  - Each process waiting for the other to signal
- Starvation: indefinite blocking
  - Process is never removed from the semaphore queue in which its suspended
  - May be caused by ordering in queues (priority)

Shared variables

```
Semaphore A(1),B(1);
```

Process P_0

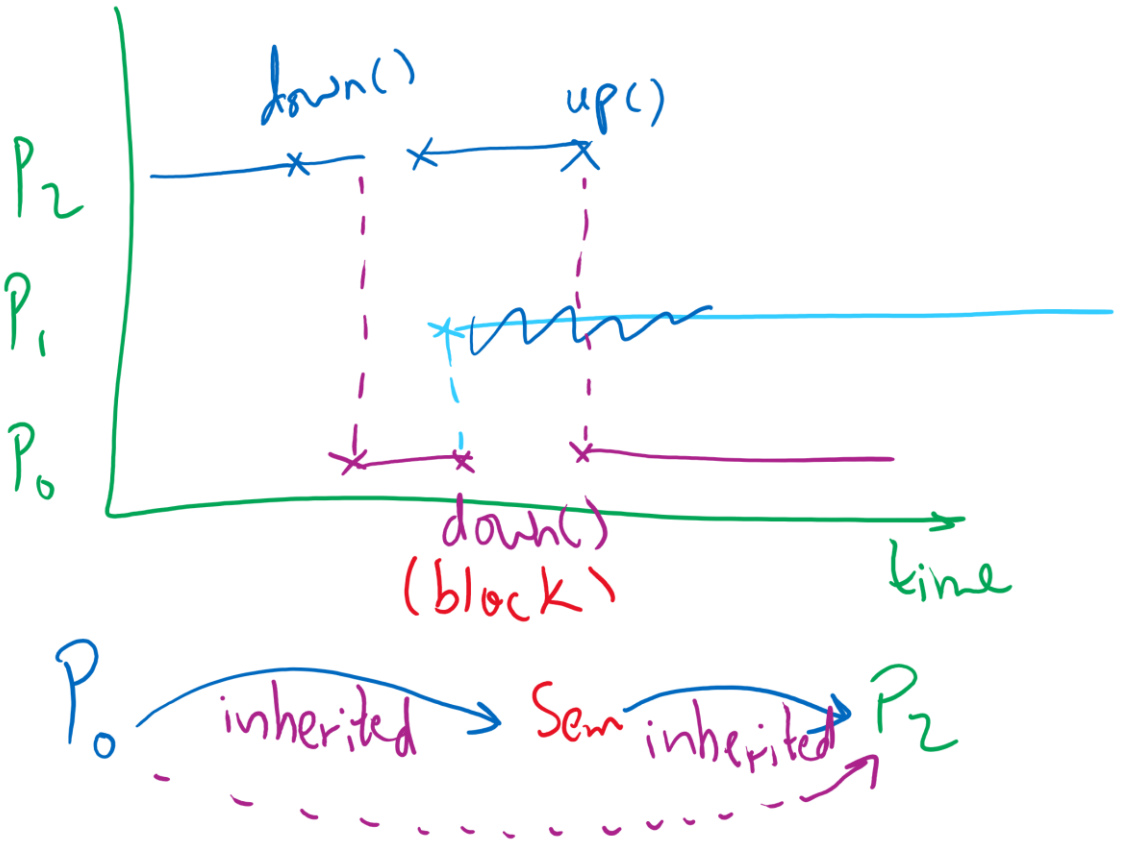
```
A.down();
B.down();
    .
    .
    .
B.up();
A.up();
```

Process P_1

```
B.down();
A.down();
    .
    .
    .
A.up();
B.up();
```

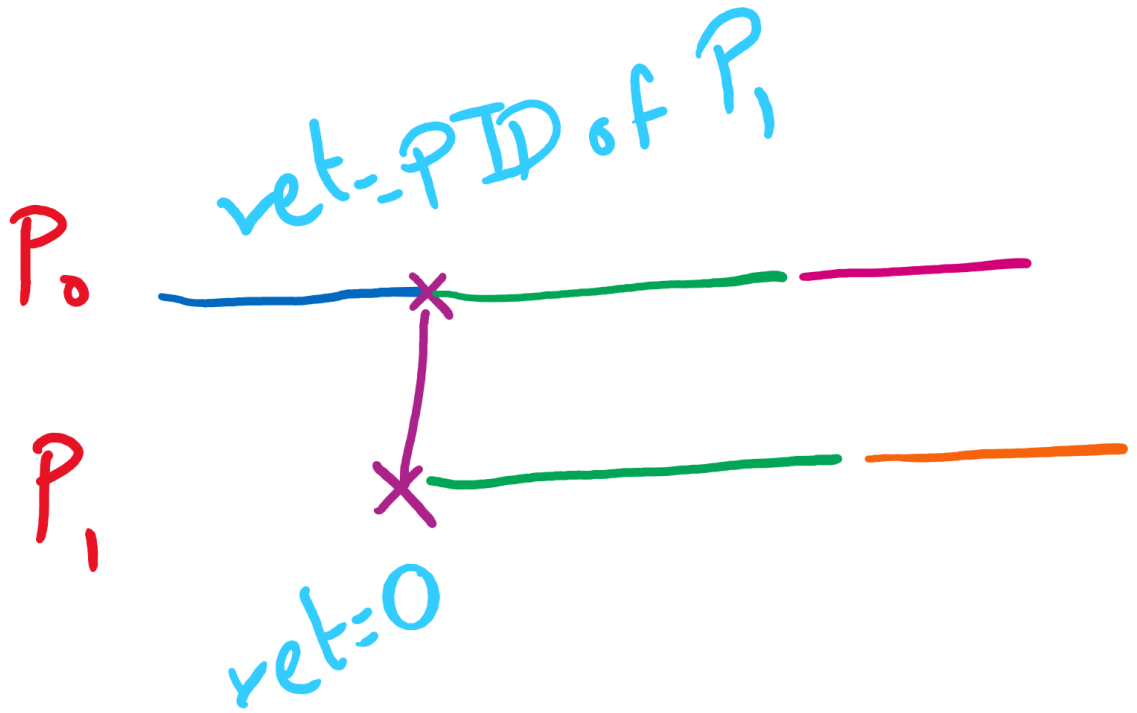
Semaphore Usage Problem: Priority Inversion

- Priority inversion is still possible using semaphores
  - Slightly less likely
  - Needs at least three processes



fork() tracing

```
int ret= fork();
if (ret==0)
if (ret >0)
```



fork()'s of fork()'s

